

B37VB Praxis Programming Game Project Report

Jack McGregor

Introduction

The aim of this project was to develop a game in C with the help of the Raylib graphics library. I decided to create a simple dodge the asteroids style game, in which the score increases the longer the player goes without collision with an asteroid, there are milestones in the score which cause the speed of the asteroids to increase. The game is made up of concepts such as variables, arrays and for loops. This report will cover the design process, rules and structure of my game.

Design

The game is made up of 3 different screens, The start screen, gameplay screen and game over screen. All of these are contained in a 600 by 600 window. This was chosen to allow for easier calculations when centring text and graphics. The wireframes for these screens are shown below.

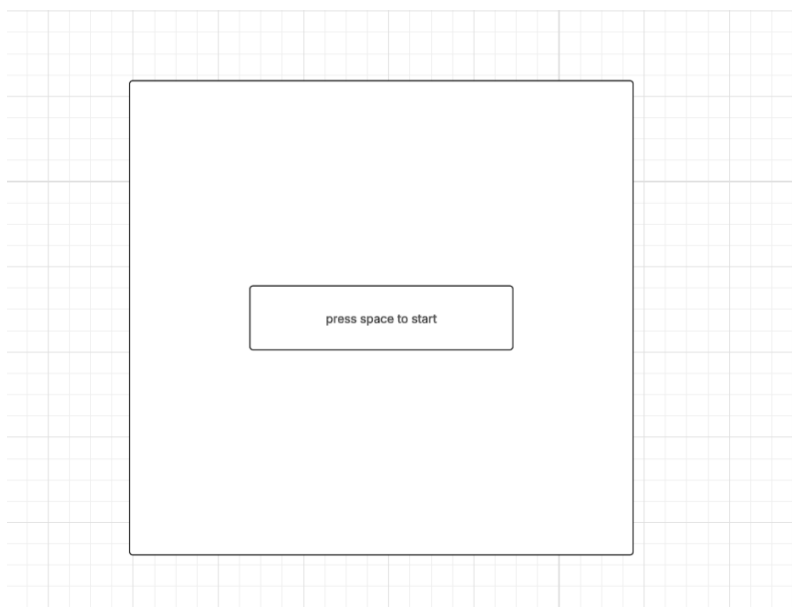


Figure 1 Wireframe of start screen

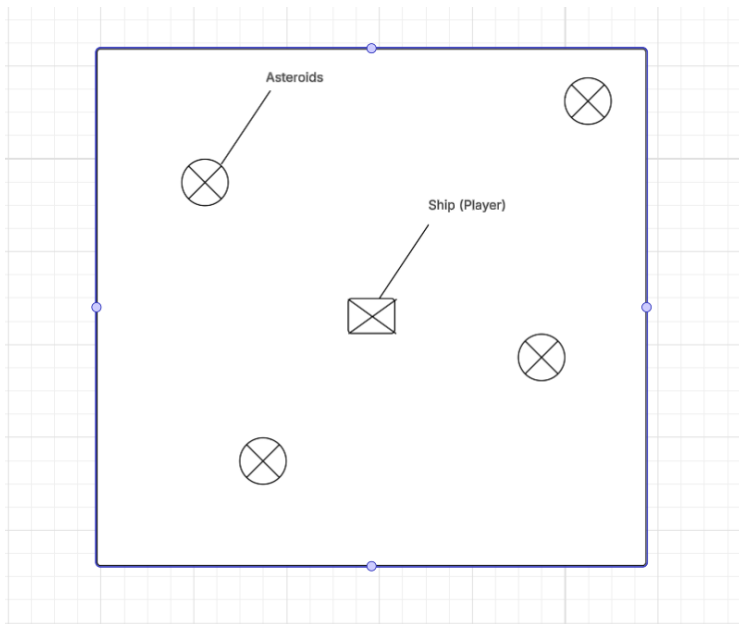


Figure 2 Wireframe of gameplay screen

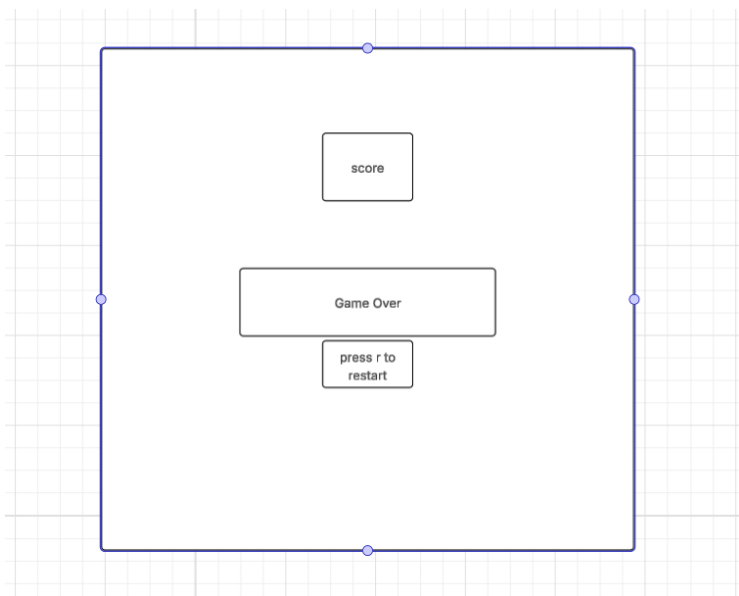


Figure 3 Wireframe of Game over screen

The final game has slight variations from these wireframes as it was decided the score should be displayed on the gameplay screen as well and its position was moved on the game over screen. The game was originally created using simple triangle and circle shapes for the player and objects, these were created using the raylib `DrawTriangle()` and `DrawCircle()` functions. These were later replaced by sprites created by me using the website piskelapp.com. These sprites are shown below.

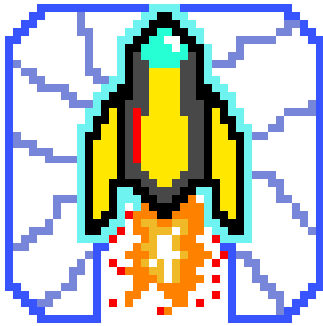


Figure 4 Ship Sprite

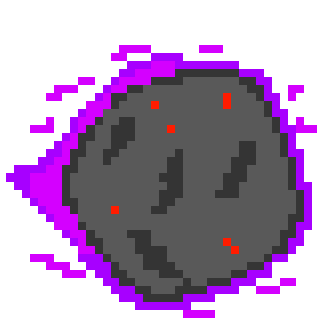


Figure 5 Asteroid sprite 1

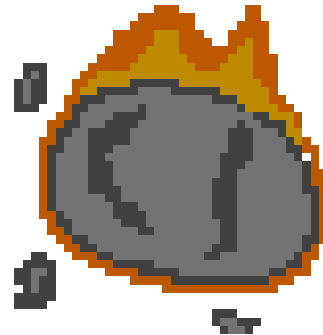


Figure 6 Asteroid sprite 2

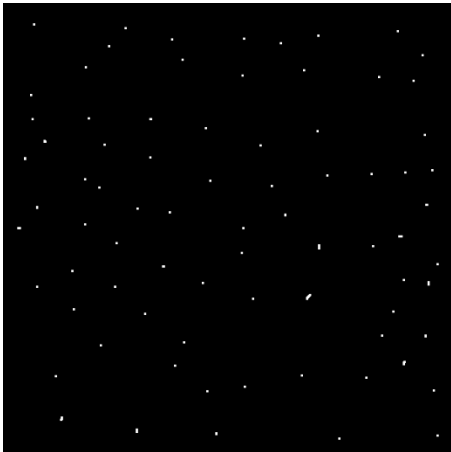


Figure 7 Background 1

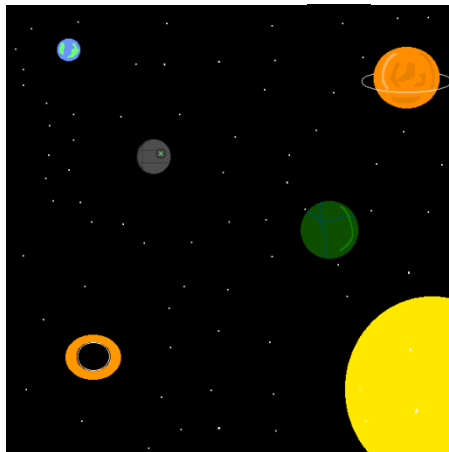


Figure 8 Background 2

The rules of the game are quite simple. The player moves the ship up down left and right using the WASD keys whilst attempting to dodge asteroids falling from the top of the screen and travelling right from the left side of the screen. The longer the player avoids a collision the higher the score.

Development

The game was developed using the Raylib graphics library. It runs by calling the `main(void)` function which then calls multiple written functions that control different aspects of the game. Inside of `main` the first function called is the Raylib function `InitWindow(600,600, "Asteroid game");` which creates the window that the game is displayed in. 600 by 600 refers to the size of the window which the game is displayed in. 600 by 600 was chosen to make it easier to calculate the positioning of objects. Next the target frames per second was set at 60, as this ensures the speed of the ship and asteroids are the same on all devices. The main game loop consists of checking if the start key is pressed, then changing to the gameplay screen, where the ship position, asteroid position and both their hitboxes are constantly updated. The `graphics()` function then uses these updated values to redraw the screen 60 times every second.

This includes the ship, asteroids, score and background. Once the score surpasses a milestone the speed of asteroids increases. If a collision is detected the screen will change to the game over screen, where the score is displayed, until the R key is pressed, at which point the game will reset the score and position of the ship and asteroids and then restart. A collision is detected when the hitboxes of the ship and an asteroid overlap.

The asteroids are stored in two arrays, one for vertical asteroids and one for horizontal asteroids. This way the position and hitboxes of asteroids can be easily updated using a for loop rather than individually writing code to update every asteroid.

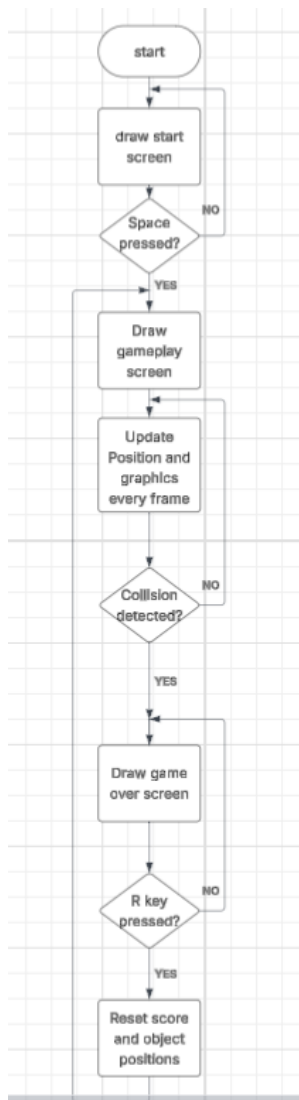


Figure 9 flowchart of main game loop

Shown below is the earliest screenshot in the development of the game.

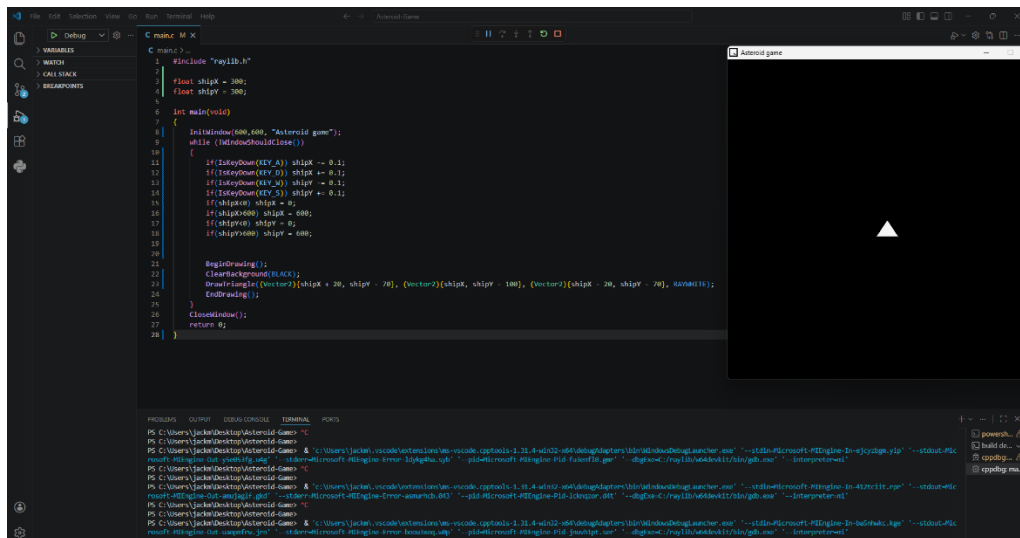


Figure 10 Early screenshot of game

In the beginning the ships movement was originally controlled in the main function, this was later changed to its own function which is called by main. The ship was represented by a triangle which was later changed to a designed sprite.

Shown below is a screenshot of the finished game

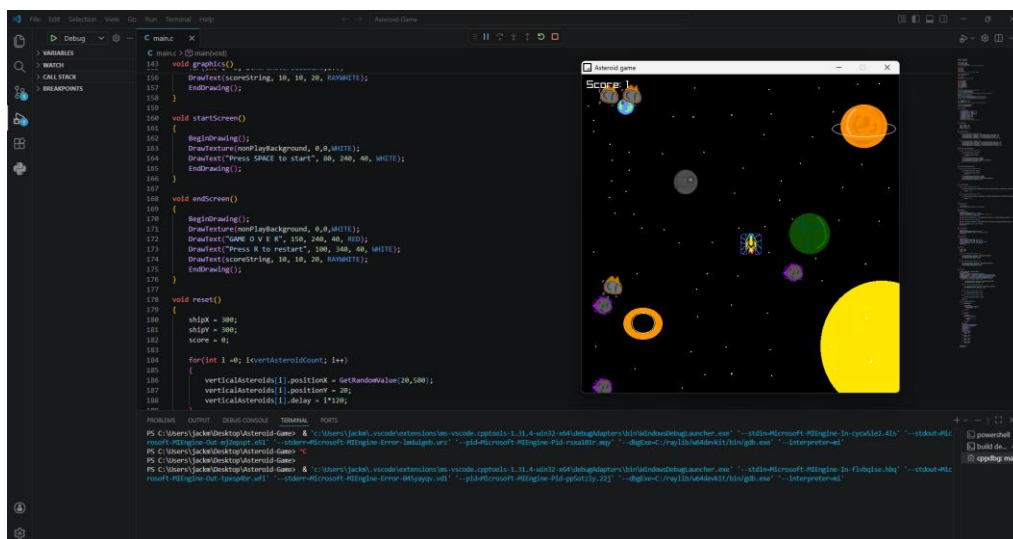


Figure 11 Screenshot of final game

Possible improvements

Some possible improvements that could be made would be, having asteroids travel at different speeds and from alternating directions. Or by having the number of asteroids increase as the game progresses as a way of increasing difficulty. A high score display could also be implemented.